

对 folders 进行第二遍扫描来添加交换过的 Message:

```
524 void swap(Message &lhs, Message &rhs)
    {
        using std::swap; // 在本例中严格来说并不需要, 但这是一个好习惯
        // 将每个消息的指针从它 (原来) 所在 Folder 中删除
        for (auto f: lhs.folders)
            f->remMsg(&lhs);
        for (auto f: rhs.folders)
            f->remMsg(&rhs);
        // 交换 contents 和 Folder 指针 set
        swap(lhs.folders, rhs.folders); // 使用 swap(set&, set&)
        swap(lhs.contents, rhs.contents); // swap(string&, string&)
        // 将每个 Message 的指针添加到它的 (新) Folder 中
        for (auto f: lhs.folders)
            f->addMsg(&lhs);
        for (auto f: rhs.folders)
            f->addMsg(&rhs);
    }
```

### 13.4 节练习

**练习 13.33:** 为什么 Message 的成员 save 和 remove 的参数是一个 Folder&? 为什么我们不将参数定义为 Folder 或是 const Folder&?

**练习 13.34:** 编写本节所描述的 Message。

**练习 13.35:** 如果 Message 使用合成的拷贝控制成员, 将会发生什么?

**练习 13.36:** 设计并实现对应的 Folder 类。此类应该保存一个指向 Folder 中包含的 Message 的 set。

**练习 13.37:** 为 Message 类添加成员, 实现向 folders 添加或删除一个给定的 Folder\*。这两个成员类似 Folder 类的 addMsg 和 remMsg 操作。

**练习 13.38:** 我们并未使用拷贝和交换方式来设计 Message 的赋值运算符。你认为其原因是什么?



## 13.5 动态内存管理类

某些类需要在运行时分配可变大小的内存空间。这种类通常可以 (并且如果它们确实可以的话, 一般应该) 使用标准库容器来保存它们的数据。例如, 我们的 StrBlob 类使用一个 vector 来管理其元素的底层内存。

但是, 这一策略并不是对每个类都适用; 某些类需要自己进行内存分配。这些类一般来说必须定义自己的拷贝控制成员来管理所分配的内存。

525 例如, 我们将实现标准库 vector 类的一个简化版本。我们所做的一个简化是不使用模板, 我们的类只用于 string。因此, 它被命名为 StrVec。

### StrVec 类的设计

回忆一下, vector 类将其元素保存在连续内存中。为了获得可接受的性能, vector

预先分配足够的内存来保存可能需要的更多元素（参见 9.4 节，第 317 页）。vector 的每个添加元素的成员函数会检查是否有空间容纳更多的元素。如果有，成员函数会在下一个可用位置构造一个对象。如果没有可用空间，vector 就会重新分配空间：它获得新的空间，将已有元素移动到新空间中，释放旧空间，并添加新元素。

我们在 StrVec 类中使用类似的策略。我们将使用一个 allocator 来获得原始内存（参见 12.2.2 节，第 427 页）。由于 allocator 分配的内存是未构造的，我们将在需要添加新元素时用 allocator 的 construct 成员在原始内存中创建对象。类似的，当我们删除一个元素时，我们将使用 destroy 成员来销毁元素。

每个 StrVec 有三个指针成员指向其元素所使用的内存：

- elements，指向分配的内存中的首元素
- first\_free，指向最后一个实际元素之后的位置
- cap，指向分配的内存末尾之后的位置

图 13.2 说明了这些指针的含义。



图 13.2: StrVec 内存分配策略

除了这些指针之外，StrVec 还有一个名为 alloc 的静态成员，其类型为 allocator<string>。alloc 成员会分配 StrVec 使用的内存。我们的类还有 4 个工具函数：

- alloc\_n\_copy 会分配内存，并拷贝一个给定范围中的元素。
- free 会销毁构造的元素并释放内存。
- chk\_n\_alloc 保证 StrVec 至少有容纳一个新元素的空间。如果没有空间添加新元素，chk\_n\_alloc 会调用 reallocate 来分配更多内存。
- reallocate 在内存用完时为 StrVec 分配新内存。

虽然我们关注的是类的实现，但我们也将定义 vector 接口中的一些成员。

## StrVec 类定义

526

有了上述实现概要，我们现在可以定义 StrVec 类，如下所示：

```
// 类 vector 类内存分配策略的简化实现
class StrVec {
public:
    StrVec(): // allocator 成员进行默认初始化
        elements(nullptr), first_free(nullptr), cap(nullptr) { }
    StrVec(const StrVec&); // 拷贝构造函数
    StrVec &operator=(const StrVec&); // 拷贝赋值运算符
    ~StrVec(); // 析构函数
    void push_back(const std::string&); // 拷贝元素
    size_t size() const { return first_free - elements; }
    size_t capacity() const { return cap - elements; }
    std::string *begin() const { return elements; }
    std::string *end() const { return first_free; }
```

```

// ...
private:
    Static std::allocator<std::string> alloc; // 分配元素
    // 被添加元素的函数所使用
    void chk_n_alloc()
    { if (size() == capacity()) reallocate(); }
    // 工具函数, 被拷贝构造函数、赋值运算符和析构函数所使用
    std::pair<std::string*, std::string*> alloc_n_copy
        (const std::string*, const std::string*);
    void free(); // 销毁元素并释放内存
    void reallocate(); // 获得更多内存并拷贝已有元素
    std::string *elements; // 指向数组首元素的指针
    std::string *first_free; // 指向数组第一个空闲元素的指针
    std::string *cap; // 指向数组尾后位置的指针
};

```

类体定义了多个成员:

- 默认构造函数(隐式地)默认初始化 alloc 并(显式地)将指针初始化为 nullptr, 表明没有元素。
- size 成员返回当前真正正在使用的元素的数目, 等于 first\_free-elements。
- capacity 成员返回 StrVec 可以保存的元素的数量, 等价于 cap-elements。
- 当没有空间容纳新元素, 即 cap==first\_free 时, chk\_n\_alloc 会为 StrVec 重新分配内存。
- begin 和 end 成员分别返回指向首元素(即 elements)和最后一个构造的元素之后位置(即 first\_free)的指针。

## 使用 construct

函数 push\_back 调用 chk\_n\_alloc 确保有空间容纳新元素。如果需要, 527 chk\_n\_alloc 会调用 reallocate。当 chk\_n\_alloc 返回时, push\_back 知道必有空间容纳新元素。它要求其 allocator 成员来 construct 新的尾元素:

```

void StrVec::push_back(const string& s)
{
    chk_n_alloc(); // 确保有空间容纳新元素
    // 在 first_free 指向的元素中构造 s 的副本
    alloc.construct(first_free++, s);
}

```

当我们用 allocator 分配内存时, 必须记住内存是未构造的(参见 12.2.2 节, 第 428 页)。为了使用此原始内存, 我们必须调用 construct, 在此内存中构造一个对象。传递给 construct 的第一个参数必须是一个指针, 指向调用 allocate 所分配的未构造的内存空间。剩余参数确定用哪个构造函数来构造对象。在本例中, 只有一个额外参数, 类型为 string, 因此会使用 string 的拷贝构造函数。

值得注意的是, 对 construct 的调用也会递增 first\_free, 表示已经构造了一个新元素。它使用前置递增(参见 4.5 节, 第 131 页), 因此这个调用会在 first\_free 当前值指定的地址构造一个对象, 并递增 first\_free 指向下一个未构造的元素。

### alloc\_n\_copy 成员

我们在拷贝或赋值 `StrVec` 时，可能会调用 `alloc_n_copy` 成员。类似 `vector`，我们的 `StrVec` 类有类值的行为（参见 13.2.1 节，第 453 页）。当我们拷贝或赋值 `StrVec` 时，必须分配独立的内存，并从原 `StrVec` 对象拷贝元素至新对象。

`alloc_n_copy` 成员会分配足够的内存来保存给定范围的元素，并将这些元素拷贝到新分配的内存中。此函数返回一个指针的 `pair`（参见 11.2.3 节，第 379 页），两个指针分别指向新空间的开始位置和拷贝的尾后的位置：

```
pair<string*, string*>
StrVec::alloc_n_copy(const string *b, const string *e)
{
    // 分配空间保存给定范围中的元素
    auto data = alloc.allocate(e - b);
    // 初始化并返回一个 pair，该 pair 由 data 和 uninitialized_copy 的返回值构成
    return {data, uninitialized_copy(b, e, data)};
}
```

`alloc_n_copy` 用尾后指针减去首元素指针，来计算需要多少空间。在分配内存之后，它必须在此空间中构造给定元素的副本。

它是在返回语句中完成拷贝工作的，返回语句中对返回值进行了列表初始化（参见 6.3.2 节，第 203 页）。返回的 `pair` 的 `first` 成员指向分配的内存的开始位置；`second` 成员则是 `uninitialized_copy`（参见 12.2.2 节，第 429 页）的返回值，此值是一个指针，指向最后一个构造元素之后的位置。

◀ 528

### free 成员

`free` 成员有两个责任：首先 `destroy` 元素，然后释放 `StrVec` 自己分配的内存空间。for 循环调用 `allocator` 的 `destroy` 成员，从构造的尾元素开始，到首元素为止，逆序销毁所有元素：

```
void StrVec::free()
{
    // 不能传递给 deallocate 一个空指针，如果 elements 为 0，函数什么也不做
    if (elements) {
        // 逆序销毁旧元素
        for (auto p = first_free; p != elements; /* 空 */)
            alloc.destroy(--p);
        alloc.deallocate(elements, cap - elements);
    }
}
```

`destroy` 函数会运行 `string` 的析构函数。`string` 的析构函数会释放 `string` 自己分配的内存空间。

一旦元素被销毁，我们就调用 `deallocate` 来释放本 `StrVec` 对象分配的内存空间。我们传递给 `deallocate` 的指针必须是之前某次 `allocate` 调用所返回的指针。因此，在调用 `deallocate` 之前我们首先检查 `elements` 是否为空。

### 拷贝控制成员

实现了 `alloc_n_copy` 和 `free` 成员后，为我们的类实现拷贝控制成员就很简单了。

拷贝构造函数调用 `alloc_n_copy`:

```
StrVec::StrVec(const StrVec &s)
{
    // 调用 alloc_n_copy 分配空间以容纳与 s 中一样多的元素
    auto newdata = alloc_n_copy(s.begin(), s.end());
    elements = newdata.first;
    first_free = cap = newdata.second;
}
```

并将返回结果赋予数据成员。`alloc_n_copy` 的返回值是一个指针的 `pair`。其 `first` 成员指向第一个构造的元素，`second` 成员指向最后一个构造的元素之后的位置。由于 `alloc_n_copy` 分配的空间恰好容纳给定的元素，`cap` 也指向最后一个构造的元素之后的位置。

析构造函数调用 `free`:

```
StrVec::~~StrVec() { free(); }
```

拷贝赋值运算符在释放已有元素之前调用 `alloc_n_copy`，这样就可以正确处理自赋值了:

529

```
StrVec &StrVec::operator=(const StrVec &rhs)
{
    // 调用 alloc_n_copy 分配内存，大小与 rhs 中元素占用空间一样多
    auto data = alloc_n_copy(rhs.begin(), rhs.end());
    free();
    elements = data.first;
    first_free = cap = data.second;
    return *this;
}
```

类似拷贝构造函数，拷贝赋值运算符使用 `alloc_n_copy` 的返回值来初始化它的指针。



### 在重新分配内存的过程中移动而不是拷贝元素

在编写 `reallocate` 成员函数之前，我们稍微思考一下此函数应该做什么。它应该

- 为一个新的、更大的 `string` 数组分配内存
- 在内存空间的前一部分构造对象，保存现有元素
- 销毁原内存空间中的元素，并释放这块内存

观察这个操作步骤，我们可以看出，为一个 `StrVec` 重新分配内存空间会引起从旧内存空间到新内存空间逐个拷贝 `string`。虽然我们不知道 `string` 的实现细节，但我们知道 `string` 具有类值行为。当拷贝一个 `string` 时，新 `string` 和原 `string` 是相互独立的。改变原 `string` 不会影响到副本，反之亦然。

由于 `string` 的行为类似值，我们可以得出结论，每个 `string` 对构成它的所有字符都会保存自己的一份副本。拷贝一个 `string` 必须为这些字符分配内存空间，而销毁一个 `string` 必须释放所占用的内存。

拷贝一个 `string` 就必须真的拷贝数据，因为通常情况下，在我们拷贝了一个 `string` 之后，它就会有用户。但是，如果是 `reallocate` 拷贝 `StrVec` 中的 `string`，则在拷贝之后，每个 `string` 只有唯一的用户。一旦将元素从旧空间拷贝到了新空间，我们就会立即销毁原 `string`。

因此，拷贝这些 `string` 中的数据是多余的。在重新分配内存空间时，如果我们能避免分配和释放 `string` 的额外开销，`StrVec` 的性能会好得多。

### 移动构造函数和 `std::move`

通过使用新标准库引入的两种机制，我们就可以避免 `string` 的拷贝。首先，有一些标准库类，包括 `string`，都定义了所谓的“移动构造函数”。关于 `string` 的移动构造函数如何工作的细节，以及有关实现的任何其他细节，目前都尚未公开。但是，我们知道，移动构造函数通常是将资源从给定对象“移动”而不是拷贝到正在创建的对象。而且我们知道标准库保证“移后源”（`moved-from`）`string` 仍然保持一个有效的、可析构的状态。对于 `string`，我们可以想象每个 `string` 都有一个指向 `char` 数组的指针。可以假定 `string` 的移动构造函数进行了指针的拷贝，而不是为字符分配内存空间然后拷贝字符。

C++  
11

◀ 530

我们使用的第二个机制是一个名为 `move` 的标准库函数，它定义在 `utility` 头文件中。目前，关于 `move` 我们需要了解两个关键点。首先，当 `reallocate` 在新内存中构造 `string` 时，它必须调用 `move` 来表示希望使用 `string` 的移动构造函数，原因我们将在 13.6.1 节（第 470 页）中解释。如果它漏掉了 `move` 调用，将会使用 `string` 的拷贝构造函数。其次，我们通常不为 `move` 提供一个 `using` 声明（参见 3.1 节，第 74 页），原因将在 18.2.3 节（第 706 页）中解释。当我们使用 `move` 时，直接调用 `std::move` 而不是 `move`。

### `reallocate` 成员

了解了这些知识，现在就可以编写 `reallocate` 成员了。首先调用 `allocate` 分配新内存空间。我们每次重新分配内存时都会将 `StrVec` 的容量加倍。如果 `StrVec` 为空，我们将分配容纳一个元素的空间：

```
void StrVec::reallocate()
{
    // 我们将分配当前大小两倍的内存空间
    auto newcapacity = size() ? 2 * size() : 1;
    // 分配新内存
    auto newdata = alloc.allocate(newcapacity);
    // 将数据从旧内存移动到新内存
    auto dest = newdata;           // 指向新数组中下一个空闲位置
    auto elem = elements;         // 指向旧数组中下一个元素
    for (size_t i = 0; i != size(); ++i)
        alloc.construct(dest++, std::move(*elem++));
    free(); // 一旦我们移动完元素就释放旧内存空间
    // 更新我们的数据结构，执行新元素
    elements = newdata;
    first_free = dest;
    cap = elements + newcapacity;
}
```

`for` 循环遍历每个已有元素，并在新内存空间中 `construct` 一个对应元素。我们使用 `dest` 指向构造新 `string` 的内存，使用 `elem` 指向原数组中的元素。我们每次用后置递增运算将 `dest`（和 `elem`）推进到各自数组中的下一个元素。

`construct` 的第二个参数（即，确定使用哪个构造函数的参数（参见 12.2.2 节，第 428 页））是 `move` 返回的值。调用 `move` 返回的结果会令 `construct` 使用 `string` 的移

动构造函数。由于我们使用了移动构造函数，这些 `string` 管理的内存将不会被拷贝。相反，我们构造的每个 `string` 都会从 `elem` 指向的 `string` 那里接管内存的所有权。

531

在元素移动完毕后，我们调用 `free` 销毁旧元素并释放 `StrVec` 原来使用的内存。`string` 成员不再管理它们曾经指向的内存；其数据的管理职责已经转移给新 `StrVec` 内存中的元素了。我们不知道旧 `StrVec` 内存中的 `string` 包含什么值，但我们保证对它们执行 `string` 的析构函数是安全的。

剩下的就是更新指针，指向新分配并已初始化过的数组了。`first_free` 和 `cap` 指针分别被设置为指向最后一个构造的元素之后的位置及指向新分配空间的尾后位置。

### 13.5 节练习

**练习 13.39:** 编写你自己版本的 `StrVec`，包括自己版本的 `reserve`、`capacity`（参见 9.4 节，第 318 页）和 `resize`（参见 9.3.5 节，第 314 页）。

**练习 13.40:** 为你的 `StrVec` 类添加一个构造函数，它接受一个 `initializer_list<string>` 参数。

**练习 13.41:** 在 `push_back` 中，我们为什么在 `construct` 调用中使用前置递增运算？如果使用后置递增运算的话，会发生什么？

**练习 13.42:** 在你的 `TextQuery` 和 `QueryResult` 类（参见 12.3 节，第 431 页）中用你的 `StrVec` 类代替 `vector<string>`，以此来测试你的 `StrVec` 类。

**练习 13.43:** 重写 `free` 成员，用 `for_each` 和 `lambda`（参见 10.3.2 节，第 346 页）来代替 `for` 循环 `destroy` 元素。你更倾向于哪种实现，为什么？

**练习 13.44:** 编写标准库 `string` 类的简化版本，命名为 `String`。你的类应该至少有一个默认构造函数和一个接受 C 风格字符串指针参数的构造函数。使用 `allocator` 为你的 `String` 类分配所需内存。

## 13.6 对象移动

新标准的一个最主要的特性是可以移动而非拷贝对象的能力。如我们在 13.1.1 节（第 440 页）中所见，很多情况下都会发生对象拷贝。在其中某些情况下，对象拷贝后就立即被销毁了。在这些情况下，移动而非拷贝对象会大幅度提升性能。

如我们已经看到的，我们的 `StrVec` 类是这种不必要的拷贝的一个很好的例子。在重新分配内存的过程中，从旧内存将元素拷贝到新内存是不必要的，更好的方式是移动元素。使用移动而不是拷贝的另一个原因源于 `IO` 类或 `unique_ptr` 这样的类。这些类都包含不能被共享的资源（如指针或 `IO` 缓冲）。因此，这些类型的对象不能拷贝但可以移动。

532

在旧 C++ 标准中，没有直接的方法移动对象。因此，即使不必拷贝对象的情况下，我们也不得不拷贝。如果对象较大，或者是对象本身要求分配内存空间（如 `string`），进行不必要的拷贝代价非常高。类似的，在旧版本的标准库中，容器中所保存的类必须是可拷贝的。但在新标准中，我们可以用容器保存不可拷贝的类型，只要它们能被移动即可。